Secure Delivery Defect and Station Performance Dashboard

Omar Soto Palos

Master of Science in Software Engineering

Grand Canyon University

SWE-550: Software Engineering and Security Principles

Instructor: Shane Carroll

November 3, 2026

For Milestone 5, I tested the security of the delivery defect dashboard I built with Flask. My goal was to find anything I missed and make sure the security features I added in earlier milestones were still doing their job. I used fictional data and ran everything on my own computer. I checked my Python code with Bandit, used OWASP ZAP to look at the web application's responses, and used curl to double-check the security headers after I made changes. Bandit did not find any issues in the two Python files I scanned. However, ZAP found that my application was missing some important security headers. I updated app.py to add those headers and then scanned the application again in a new ZAP session. The original missing-header alerts were gone. I also learned that fixing one issue does not always mean everything is secure. ZAP reported a separate warning because my new CSP still allows inline CSS, and the server continued to show its version information.

I also checked the protected parts of the application after completing the initial scans. I logged in with my fictional manager test account through Chrome configured to use the ZAP proxy. ZAP recorded successful HTTP 200 responses for /dashboard and /manage-records. I then ran a separate, limited active scan against the dashboard starting point. That active scan completed, but the response showed a redirect instead of the dashboard content. I am separating these results because capturing an authenticated page and actively testing that page are not the same thing.

Scope, Test Environment, and Version

I tested the Flask dashboard on my Mac using SQLite and fictional records. The application includes login, session controls, and different access levels for team members, managers, and admins.

Field	Recorded value

Testing dates	November 3–4, 2026 ; HTTP response timestamps may be GMT
Target URL	http://127.0.0.1:5000
Environment	macOS; VS Code; Flask development server; local SQLite; fictional data
Python / Flask / Werkzeug	Python 3.14.3 / Flask 3.1.3 / Werkzeug 3.1.9
Repository	https://github.com/Ramo3695/Secure-Delivery-Defect-Dashboard
Repository branch / submission commit	main / 737a7fa (successful push shown in VS Code); local working branch: master
Initial baseline identifier	72d38e1 shown before later commits; not an official version tag
Scope boundary	Localhost only; initial Spider/passive scans; later manager-authenticated browser traffic and limited active scan of /dashboard. No production or Amazon internal systems.

For version tracking, I used the GitHub commit that contains my security fixes and Milestone 5 documents. I did not save a separate build number at the exact moment of each scan, so I am using the commit as my submission reference rather than saying every screenshot came from that exact checkout.

Tools and Test Method

Tool	Use	Evidence
Bandit 1.9.4	Static application security testing (SAST) of app.py and models.py	Saved terminal output and bandit-results.txt
OWASP ZAP 2.17.0	Dynamic application security testing (DAST): HTTP traffic, Spider, passive findings	Original and retest alert screenshots
curl	Manual HTTP verification through ZAP proxy and of security response headers	Header screenshots / HTTP 200 response
VS Code / Git	Code update, local execution, version control and upload	app.py and repository commit
ZAP custom active-scan policy	Directory Browsing only, Low threshold/strength; no recursion	Two requests, 100% completion, zero new alerts; limited coverage

I started my Flask application locally and used curl to send a request through ZAP on port 8080. I ran the Spider and checked the passive alerts from the original scan and the retest. Both Spider runs reached 100%, found five URLs, and added three nodes. Later, I used a separate Chrome window connected to the ZAP proxy, logged in with my fictional manager account, and opened /dashboard and /manage-records. I kept ZAP in Protected Mode and limited the scan

context to my localhost application. For the additional active test, I created a custom policy with only Directory Browsing enabled at Low threshold and Low strength, selected /dashboard as the starting point, and left recursion disabled.

I reviewed each alert to understand what ZAP was reporting. I did not treat the informational messages as security problems just because they appeared in the list. All testing stayed on my local application. The results show what I found with this testing method.

Static Analysis (SAST) Results

For the code review, I ran Bandit against app.py and models.py. It checked all lines of code and reported zero issues in the high, medium, and low categories. That was a good result, but I know it does not mean the whole application is free of security problems. Bandit checks for certain patterns in Python code, while ZAP helped me find issues with the HTTP responses that Bandit did not report. I used the command python -m bandit -r app.py models.py and saved the results in bandit-results.txt. Figure 1 shows what the scan reported.

Dynamic Analysis (DAST) Results

My first ZAP scan showed six types of alerts: two Medium, two Low, and two Informational. The two Medium findings were missing Content Security Policy and anti-clickjacking protection. The Low findings were a missing X-Content-Type-Options header and server version information appearing in the response. The other two alerts identified the login request and session management. Those were useful observations, but they were not confirmed vulnerabilities.

After updating app.py, I ran the Spider again in a new session. This time ZAP showed four alert types: one Medium, one Low, and two Informational. The three missing-header alerts were no longer there. The remaining Medium alert was about allowing unsafe-inline styles in the

CSP, and the Low alert still showed the Werkzeug and Python versions. Overall, the results improved, but I still have security work to finish.

Severity	**Original scan: alert types**	**Retest: alert types**
High	0	0
Medium	2	1
Low	2	1
Informational	2	2

These numbers count alert types, not every individual response affected. I later saw a missing X-Content-Type-Options warning for clientservices.googleapis.com in the shared ZAP session. That third-party response is outside my Flask application and is not a new dashboard finding. The authenticated browsing evidence also does not mean every restricted route was actively tested.

Authenticated Browsing and Limited Active Scan

After my first scans, I wanted to check more than just the login page. I opened Chrome through ZAP and signed in with my test-manager account. ZAP captured GET /dashboard and GET /manage-records with HTTP 200 responses. That confirmed the pages were reached while I was logged in. It did not, by itself, test every permission or security weakness on those pages.

I then created Milestone5_Limited_Dashboard_Scan. I turned off all rules except Directory Browsing under Information Gathering, set the threshold and attack strength to Low, used Protected Mode and Default Context, chose http://127.0.0.1:5000/dashboard as the starting point, and left Recurse unchecked. The active scan reached 100%, sent two GET requests, and reported zero new alerts. One random-path request returned HTTP 404; the request to /dashboard/

returned HTTP 302 with Location: /. I confirmed afterward that the correct /dashboard address returned HTTP 200 while logged in because the active-scan request was redirected, I am recording this as a completed limited active scan, not a full authenticated vulnerability assessment. The additional ZAP session also contained traffic from Google services because Chrome was using the proxy.

Findings, Risk Ratings, and Validation

ID / severity	Finding and component	Impact / validation / status
F-01 / Medium	Missing CSP header; HTTP response configuration (app.py)	Weaker restrictions on browser-loaded content; true positive; resolved original alert
F-02 / Medium	Missing anti-clickjacking header; HTTP response configuration (app.py)	Potential framed-page deception; true positive; resolved
F-03 / Low	Missing X-Content-Type-Options; HTTP response configuration (app.py)	Possibility of browser content sniffing; true positive; resolved
F-04 / Low	Werkzeug/Python versions in Server header; development server	Useful environment detail for an attacker; true positive in local environment; open

F-05 / Medium	CSP style-src permits unsafe-inline; CSP in app.py and inline CSS templates	Reduced strength of style restrictions; true positive configuration warning; open

I worked on the missing CSP and anti-clickjacking headers first because they were both rated Medium and protect how the browser handles my application. Since my dashboard includes accounts with different roles, I wanted those protections in place. I fixed the missing nosniff header in the same update because it was a straightforward improvement. The inline-style CSP warning is now my highest-priority open issue. I also need to address the server-version header before deploying the application. I verified that the reported headers were missing or present by checking the response information.

Traceability to Security Requirements and Threat Model

Finding	**Security objective / project connection**	**Related control or decision**
F-01	Output protection and limiting execution of untrusted browser content	CSP defense in depth; browser trust boundary
F-02	Prevent deceptive interaction with authenticated management pages	Anti-framing protection; spoofing / UI redress threat
F-03	Handle HTTP output predictably and limit content-type confusion	Content-Type and nosniff response configuration

F-04	Limit unnecessary technical disclosures by the application server	Production deployment hardening; information-disclosure threat
F-05	Strengthen the CSP without breaking application styles	External CSS and restrictive style-src policy; output protection

I connected the findings to the security topics I have been working on throughout the course, including protecting information, preventing unauthorized actions, and securing application responses. These are connections to my project security goals, not confirmed requirement numbers from a separate requirements document. I also understand that browser headers are only one layer of protection; my role checks still need to be enforced on the backend.

Corrective Actions and Remediation Evidence

F-01: CSP header

I added an @app.after_request function in app.py so Flask adds a Content-Security-Policy header to its responses. I limited where scripts and other resources can load from and blocked the application from being displayed inside another site's frame. The original missing-CSP alert was resolved. I did leave an exception for inline CSS so my pages would keep their formatting. ZAP flagged that exception during the retest, which is why I added it as a separate finding. Backlog item: M5-SEC-01, with follow-up under M5-SEC-05.

F-02: Anti-clickjacking headers

I also added X-Frame-Options: DENY and frame-ancestors 'none' in the CSP. These settings are meant to stop another website from showing my dashboard inside a frame and

tricking someone into clicking on it. I saw both settings in the response, and ZAP no longer reported the original clickjacking-header issue. Backlog item: M5-SEC-02.

F-03: Content-type sniffing

I added X-Content-Type-Options: nosniff to tell the browser not to guess a different content type. When I checked the updated response and reran ZAP, the original missing-header alert was gone. This was an additional fix I completed during the same update. Backlog item: M5-SEC-03.

F-04 and F-05: Planned work

Two items still need work. First, the Server header shows Werkzeug 3.1.9 and Python 3.14.3 because I am using a development server. Before putting this application into production, I would use a proper WSGI server and review the response headers again (M5-SEC-04). Second, I need to move the inline CSS into external style files and remove unsafe-inline from the CSP. After that, I would check that the login page and dashboard still look right and rerun ZAP (M5-SEC-05). I have not completed either of these fixes yet.

Retesting and Before-and-After Comparison

Finding / method	**Before**	**Change**	**After / outcome / residual risk**
F-01 / ZAP + curl	Medium: CSP header not set	Added CSP in app.py	Original missing-header alert absent: RESOLVED. New unsafe-inline CSP

			warning remains (F-05).
F-02 / ZAP + curl	Medium: X-Frame-Options absent	Added DENY and frame-ancestors none	Original alert absent: RESOLVED. No claim of full clickjacking penetration test.
F-03 / ZAP + curl	Low: nosniff header absent	Added nosniff	Original alert absent: RESOLVED. Content-type correctness requires ongoing review.
F-04 / ZAP	Low: server versions disclosed	No production server change yet	UNCHANGED. Remains open for deployment hardening.
F-05 / ZAP	Not present in original scan because CSP was missing	Introduced CSP policy with inline-style exception	NEW / OPEN. Move styles to external CSS and retest.

To retest my changes, I restarted Flask, checked its response headers with curl, and created a fresh ZAP session. I sent a request through the proxy and ran the Spider again. Then I compared the original findings with the new alert list. This gave me evidence that my security-

header changes were working. It does not replace testing the full login process or every protected route.

Risk Register and Backlog Updates

I created a risk register and a backlog for Milestone 5 so I could keep track of what I found and what I still need to fix. The risk register explains the severity, impact, current status, and remaining risk for each finding. The backlog connects those findings to actions and acceptance criteria. I marked F-01, F-02, and F-03 as resolved based on the retest. F-04 and F-05 are still open, and I assigned them follow-up work instead of marking them complete.

Backlog	**Acceptance criteria summary**	**Status**
M5-SEC-01	CSP present; original missing-CSP alert absent	Completed; additional F-05 work open
M5-SEC-02	DENY and frame-ancestors present; original alert absent	Completed
M5-SEC-03	nosniff present; original alert absent	Completed
M5-SEC-04	Appropriate production server; minimize unnecessary version disclosures	Planned before deployment
M5-SEC-05	External styles; no unsafe-inline; page rendering and ZAP retest pass	Planned next hardening iteration

Limitations, Follow-Up Testing, and Conclusion

My original ZAP Spider scans were unauthenticated, so those crawls did not cover every protected route. I improved the evidence by logging in as a test manager through ZAP and verifying HTTP 200 responses for /dashboard and /manage-records. I also completed a limited active scan, but it only sent two requests and did not return authenticated dashboard content. The request to /dashboard/ returned HTTP 302 with Location: /, while a separate random path returned HTTP 404. I cannot treat zero new alerts from those two requests as proof that the authenticated dashboard is free of vulnerabilities. I would still need more complete authenticated testing and separate admin-role checks before making that claim.

This milestone helped me see that even though I had already worked on login security and role-based access, there were still things I missed in the web responses. I fixed three missing headers and confirmed that they appeared in the updated application. I also found two issues that still need attention: the inline CSS exception in my CSP and the server version information. Rather than saying my dashboard is completely secure, I documented what passed, what did not, and what I plan to improve next.

For my next round of testing, I would restore a practical session timeout, set up a repeatable authenticated scanning configuration, confirm the correct route without a trailing slash, and separately test each role. I would also keep the scan restricted to fictional data and back up the local SQLite database before trying any broader active-scan policy.

GitHub repository: https://github.com/Ramo3695/Secure-Delivery-Defect-Dashboard. The successful VS Code push showed main updated through commit 737a7fa. I did not independently verify access to the repository from another account.

Security Testing Figures and Evidence

I included the screenshots I captured while testing so the results can be checked. I removed the visible session-cookie value from the HTTP response evidence before placing it in the report. Figures 11 through 13 show the additional limited active scan, its redirect, and successful authenticated browsing. The scan and browsing evidence support different conclusions, so I have kept them separate.

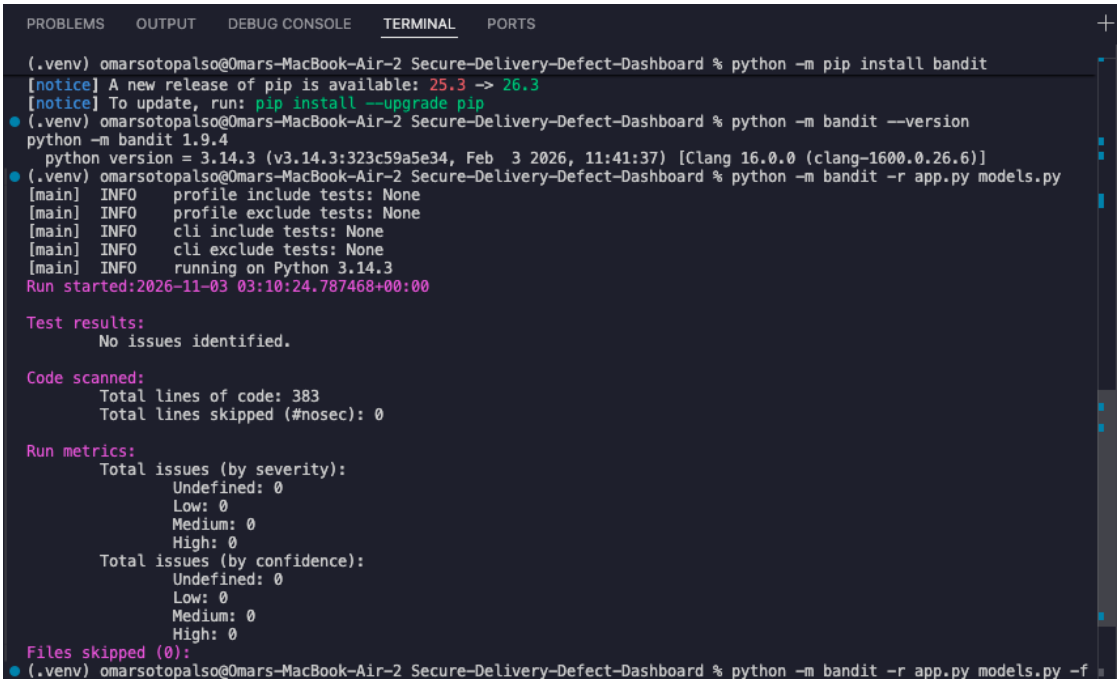


Figure 1. Bandit static analysis in the VS Code terminal. The scan of app.py and models.py reports 383 lines of code and zero issues at every listed severity.

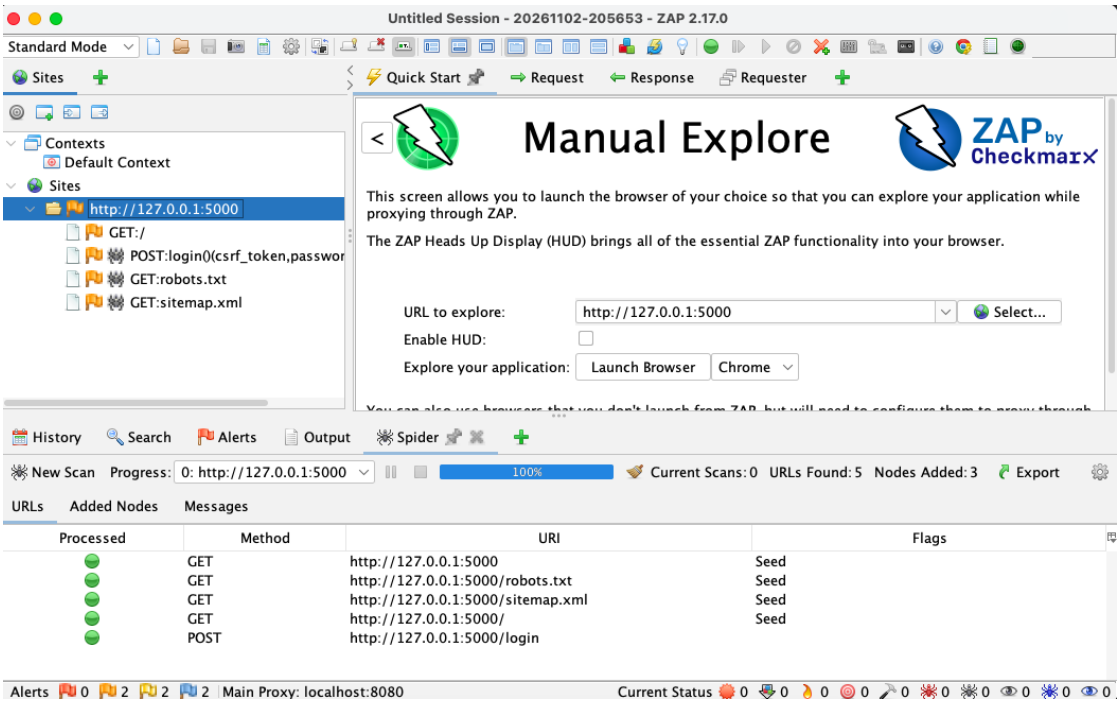


Figure 2. Original ZAP Spider progress. The crawl shows 100% completion with five URLs found and three nodes added for the localhost application.

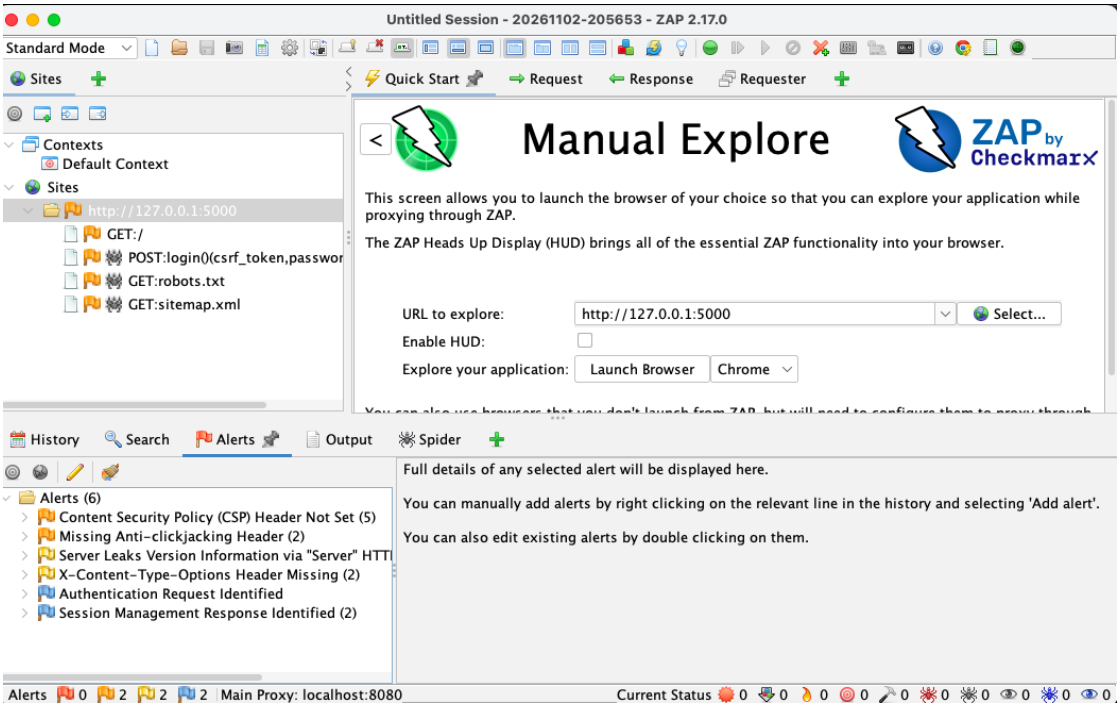


Figure 3. Original passive alert listing in ZAP. The six alert types include missing CSP, missing anti-clickjacking protection, server disclosure, missing nosniff, and two informational observations.

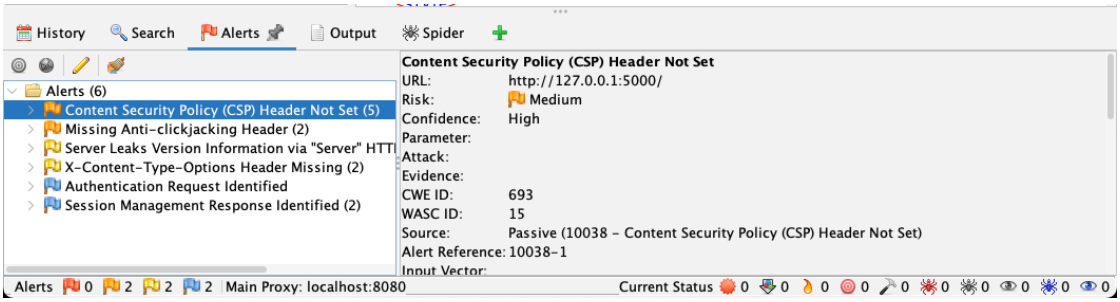


Figure 4. Original Medium-risk CSP alert. ZAP identifies the missing Content-Security-Policy response header on the local login surface.

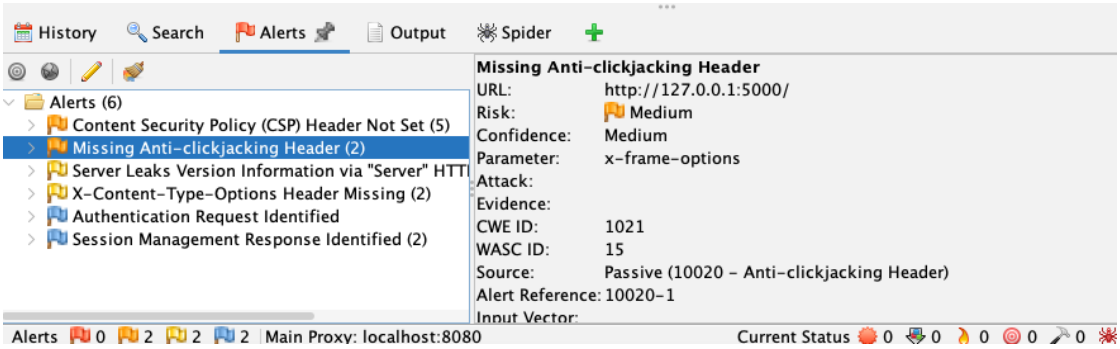


Figure 5. Original Medium-risk missing anti-clickjacking header. This response lacked the X-Frame-Options control later added in app.py.

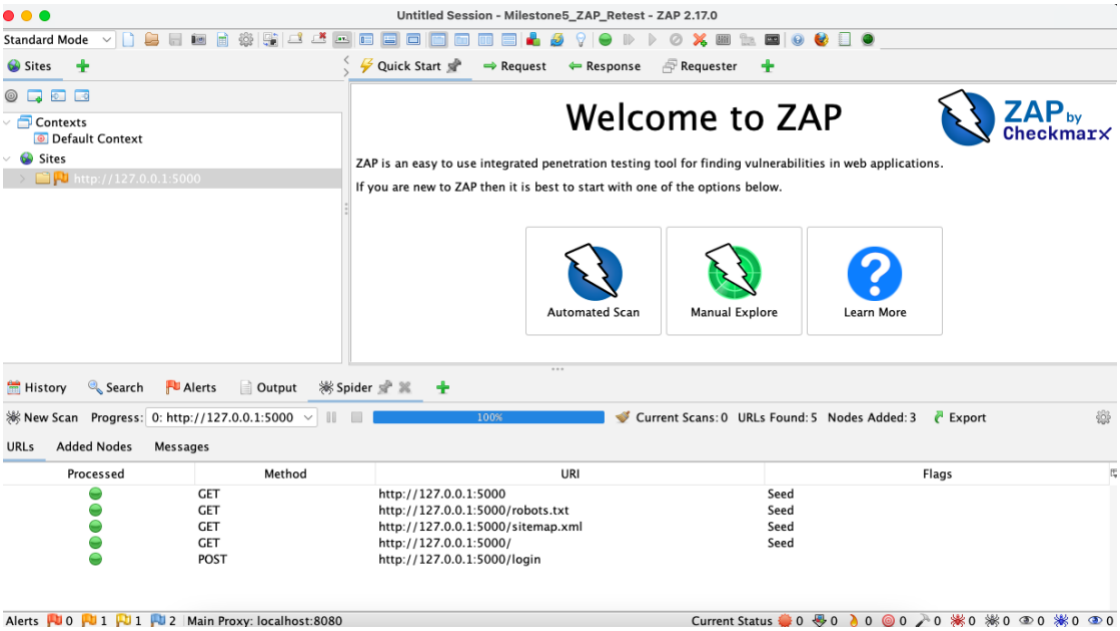


Figure 6. New-session retest Spider at 100% completion. The same localhost target was crawled again after the Flask response-header update.

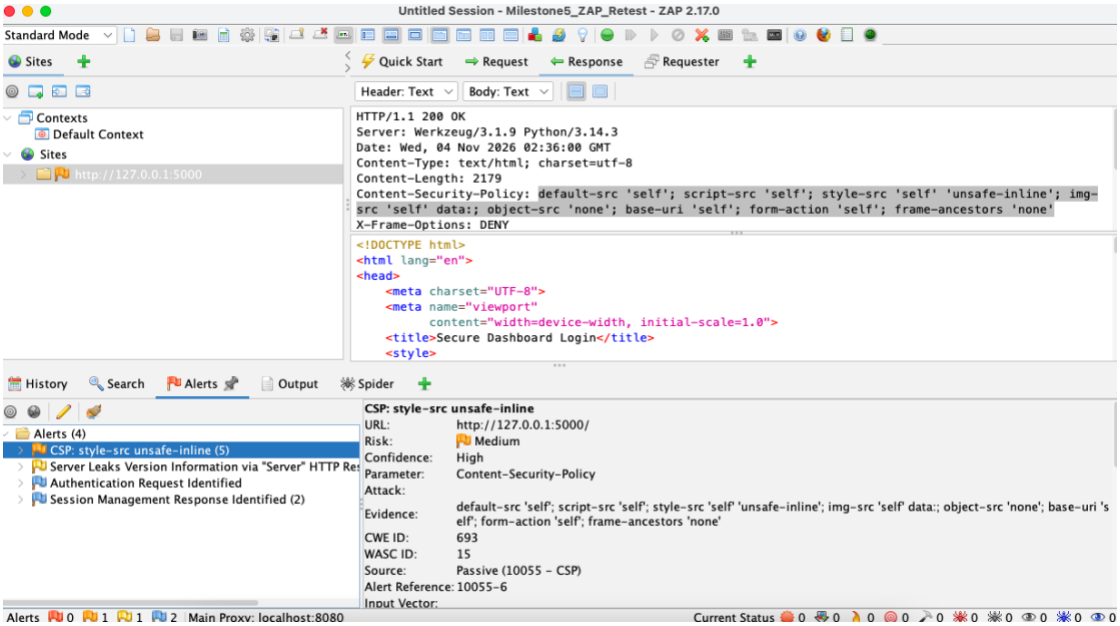


Figure 7. New retest alert list. ZAP now shows one Medium, one Low, and two Informational alert types; the original three missing-header alerts are not shown.

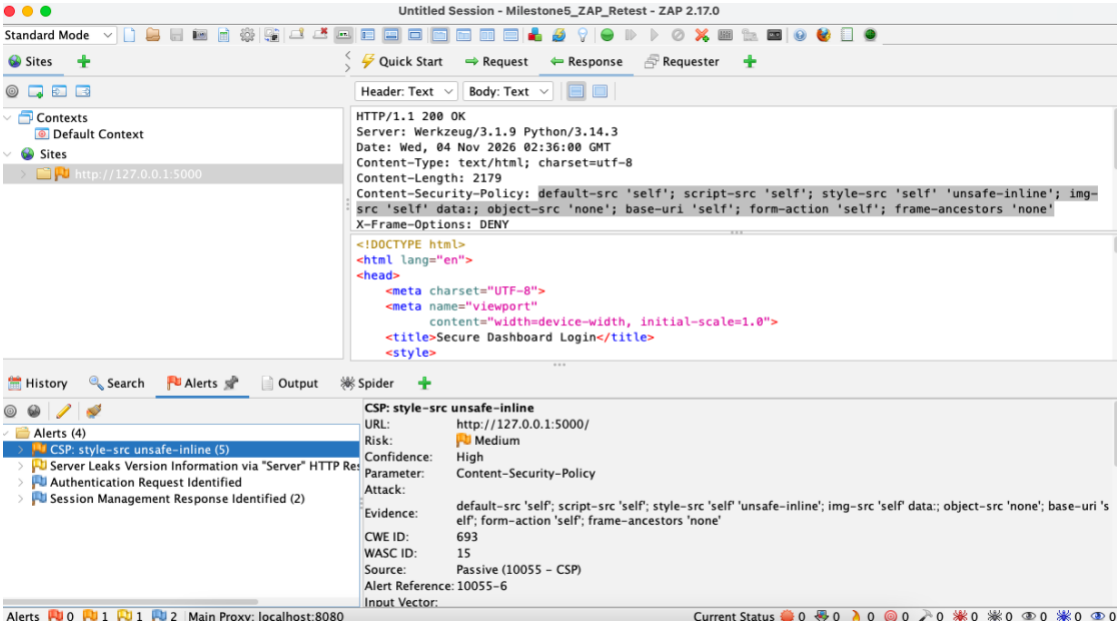


Figure 8. Newly reported Medium-risk CSP warning. ZAP highlights style-src unsafe-inline, the remaining browser-policy restriction to harden.

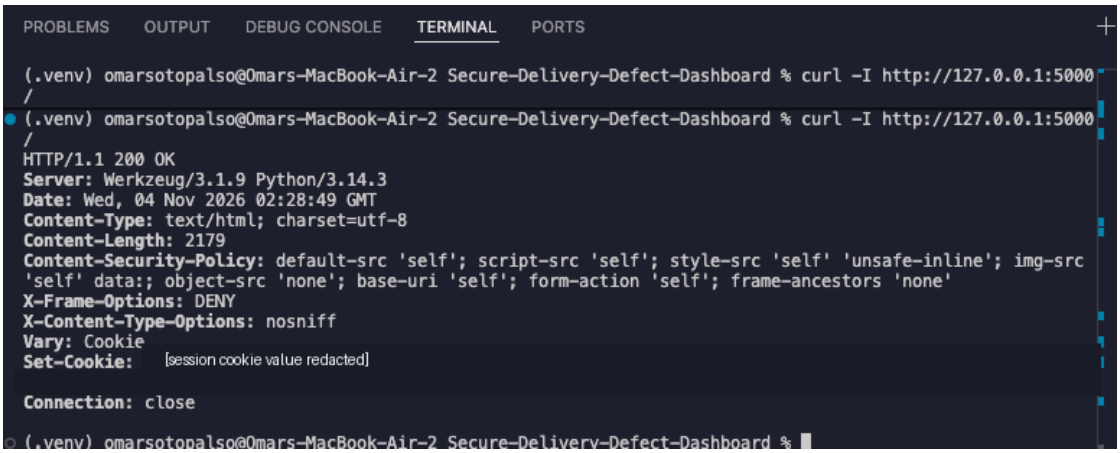


Figure 9. Manual curl response-header verification after remediation. CSP, X-Frame-Options: DENY, and X-Content-Type-Options: nosniff are shown; the session-cookie value is concealed.

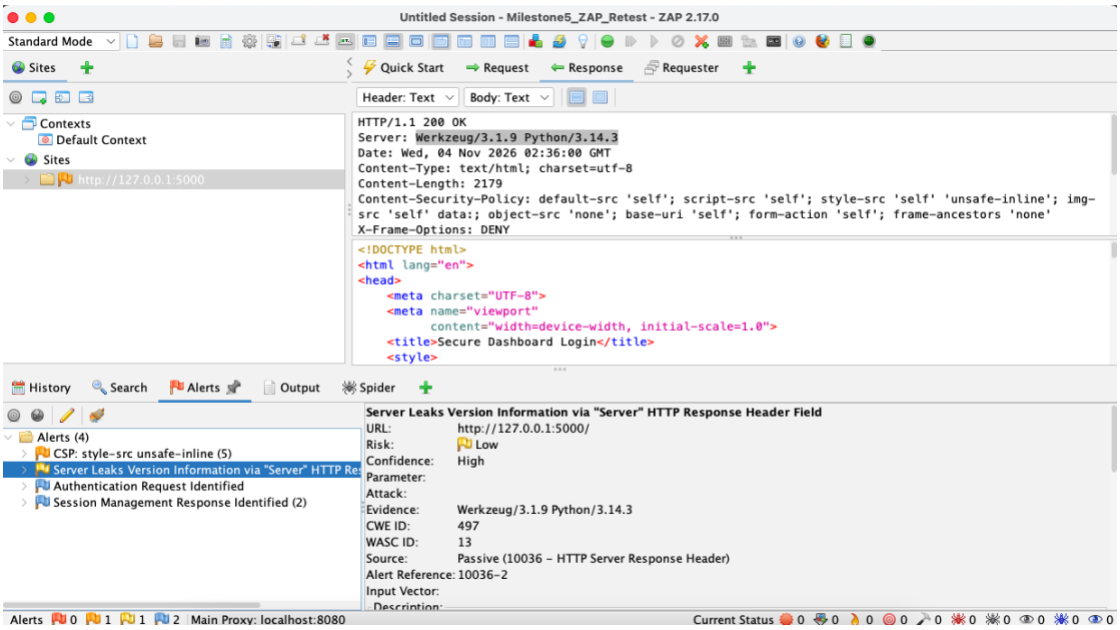


Figure 10. Persistent Low-risk server version disclosure during retest. The Server response identifies Werkzeug and Python versions, so this finding remains open.

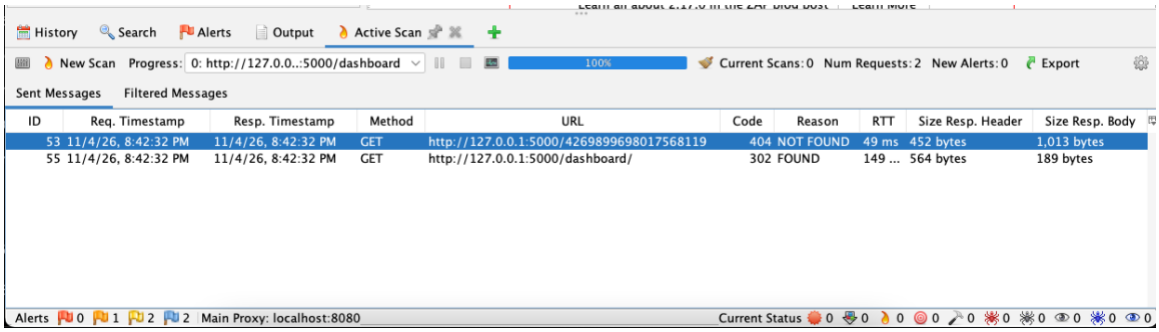


Figure 11. Limited ZAP active scan of the localhost dashboard starting point. The custom policy completed at 100% with two requests and zero new alerts; the responses shown were 404 and 302, so this does not prove full authenticated dashboard coverage.

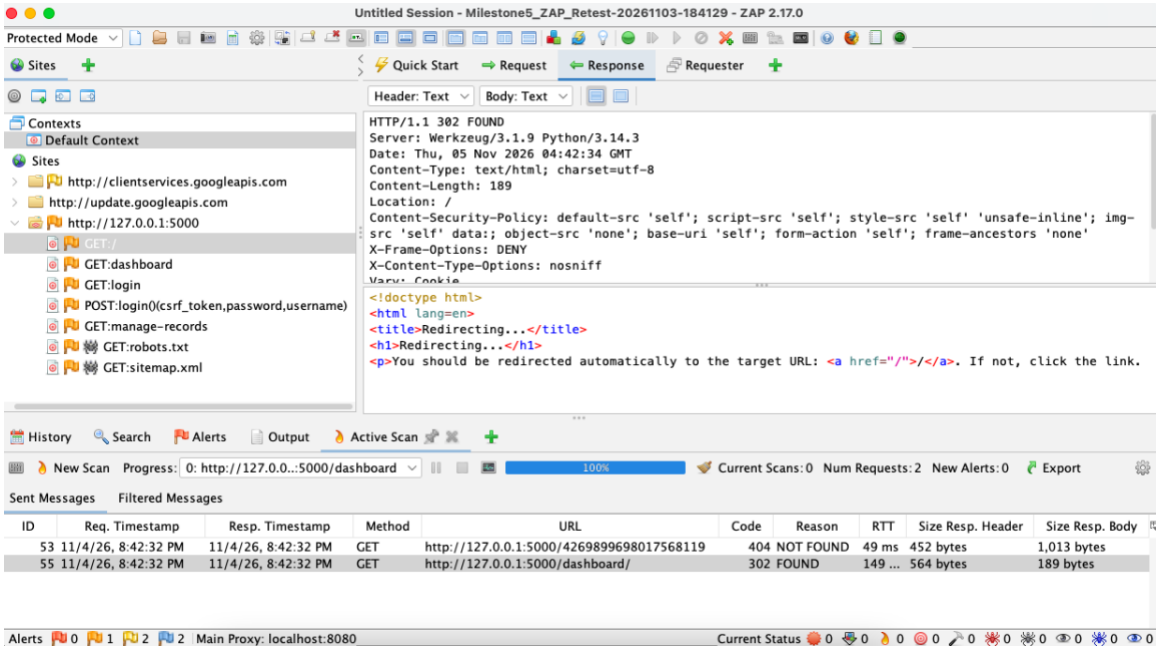


Figure 12. Active-scan redirect evidence. The GET request to /dashboard/ returned HTTP 302 with Location: /, which explains why the scan did not show a successful authenticated dashboard response.

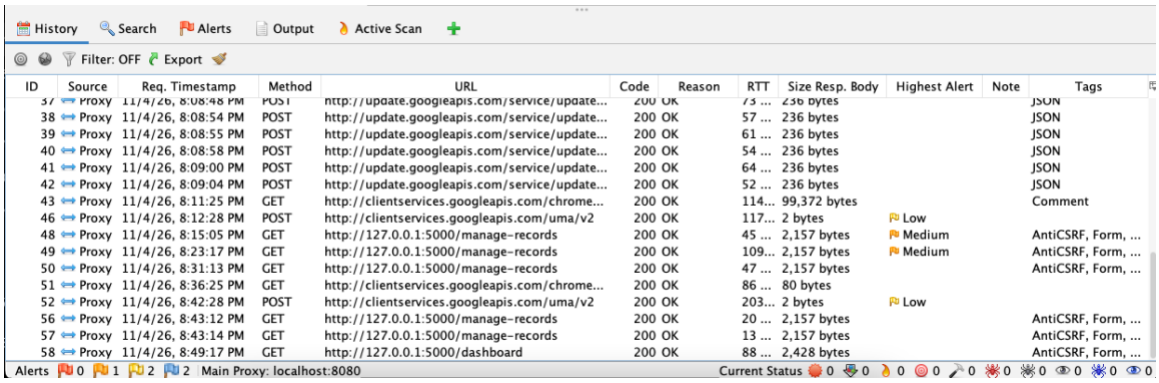


Figure 13. Authenticated browsing through the ZAP proxy. The History tab shows GET /dashboard returning HTTP 200, with manager-only /manage-records also returning 200 during the authenticated session.

Evidence and Artifact Index

Artifact	Location / use

Bandit saved report	bandit-results.txt in project root
Findings and retest log	docs/milestone-5/security-findings.md
Risk register	docs/milestone-5/risk-register.md
Remediation backlog	docs/milestone-5/backlog.md
Screenshot evidence	docs/screenshots/01-bandit-results.png through 12-zap-retest-server-disclosure.png
Application security header code	app.py - add_security_headers after_request hook